

**University of Applied Sciences and Arts
Northwestern Switzerland**

School of Computer Science

Institute for Mobile and Distributed Systems (IMVS)

Master of Science in Engineering (MSE)

Project P8

Enhancing Web Accessibility Standards

*Implementation of User-Centric Customization Interfaces and
CSS-based Contrast Optimization*

Author:	Florian Schnidrig
Advisor:	Prof. Dierk König
Profile:	Computer Science
Date:	Summer 2026 (TBD)

Contents

1. Abstract	1
2. Acknowledgement	1
3. Introduction	2
3.1. Objective of this Project	2
4. Theory	3
4.1. CSS Media Queries	3
4.1.1. Reduced Motion	4
4.1.2. Contrast	4
4.1.3. Forced Colors	4
4.1.4. Reduced Transparency	5
4.1.5. Color Scheme	5
4.1.6. Inverted Colors	5
4.1.7. Accessing Media Features through JavaScript	5
4.1.8. Changes in the World of Media Queries	6
4.1.9. Deprecated Features	6
4.1.10. Security Concerns	6
4.2. CSS Custom Properties	7
4.2.1. Global State Management	8
4.2.2. Custom Property Toggle	10
4.3. CSS Functions	12
4.3.1. CSS Custom Functions	12
4.4. Color Spaces	14
4.4.1. RGB	14
4.4.2. HSL	14
4.4.3. CIELAB Colors	15
4.4.4. HWB	15
4.5. Contrast Perception	17
4.5.1. Relative Luminance	17
4.5.2. Perceptual Color Models	18
4.6. Canvas	21
4.7. Ishihara	22
4.7.1. Functional Application in Contrast Testing	22
5. Methodology	23
6. Implementation	24
6.1. Preferences Weidget	24
6.1.1. JavaScript Part	25
6.1.2. CSS Part	28
6.2. Contrast Color Functions	31
6.3. Interactive Ishihara Plate	33
7. Discussion	34
8. Conclusions	35

Bibliography	36
List of Figures	39
List of Listings	40
List of Aids	42
Artificial Intelligence and Language Models	42

1. Abstract

2. Acknowledgement

3. Introduction

The previous project, *Accessibility on the Web – Where we Stand* [1], examined the fundamental principles of digital inclusion in modern web applications. It explored how semantic HTML forms the foundation of accessibility, provided an overview of both common and lesser-known native elements, and clarified the roles of ARIA and WCAG. In addition, it introduced the basics of accessibility debugging, testing, and the practical use of screen readers.

Building on that foundation, this project focuses on essential aspects of accessibility that may appear less technical in the traditional sense, but are equally important for creating an inclusive digital world. While the previous project concentrated primarily on the structural and technical mechanisms behind the scenes, it did not examine the visual design of web applications. These design decisions are far more than aesthetic choices. They play a decisive role in determining whether a web application becomes a welcoming space or a digital dead end for many users.

3.1. Objective of this Project

This project aims to explore accessibility in greater depth by examining ways to build on and extend existing browser standards. It also investigates additional techniques and approaches that can simplify the user journeys of people with impairments, making web applications more user-friendly and adaptable to individual needs.

4. Theory

The following chapter introduces CSS and JavaScript features, along with patterns and standards that can serve as a foundation for improving visual accessibility. These techniques help create a better user experience, particularly for people with visual impairments.

4.1. CSS Media Queries

CSS Media Queries allow developers to apply different styles based on the characteristics of the device or environment used to display a web page. This mechanism is commonly used to create responsive layouts that adapt their appearance according to the available screen width.

Listing 1 below illustrates the syntax of a media query as defined by the W3C.

```
@media <media-query-list> {  
  <rule-list>  
}
```

Listing 1: Syntax of a media query, where `<media-query-list>` contains `<media-type>` values, `<media-feature>` values, and logical operators.

In web development, the `screen` media type is the most commonly used, but media queries can also target printers by using `print`. If no media type is specified, the default value is `all`, which applies the styles to all devices.

Media features are where media queries become particularly powerful, as more than 40 features are currently available. They allow styles to respond to specific characteristics of the user's environment. Common examples include `max-width` and `max-height`, as well as their counterparts `min-width` and `min-height`. These features are widely used to create responsive websites with layouts optimized for different screen sizes.

Listing 2 demonstrates a media query that uses the `screen` media type together with the `max-width` media feature.

Some media features take accessibility needs and user preferences into account. They make it possible to adapt the presentation of a website to individual requirements. Despite their importance, they are still used less frequently than they should be. The following media features are particularly relevant when building web applications that aim to be accessible to all users. [2]

```
@media screen and (max-width: 768px) {  
  body {  
    background-color: lightskyblue;  
  }  
}
```

Listing 2: Example of a media query that changes the background color on smaller screens using the `screen` media type and the `max-width` media feature.

4.1.1. Reduced Motion

The `prefers-reduced-motion` media feature indicates whether a user has enabled a system setting that reduces non-essential motion. If the value is set to `reduce`, animations and transitions should be limited to the bare minimum.

A value of `no-preference` indicates that the user has not expressed a preference for reduced motion, allowing animations and transitions to be displayed without restriction. [3]

4.1.2. Contrast

The `prefers-contrast` media feature allows users to indicate whether they prefer content to be presented with higher or lower contrast. A value of `no-preference` indicates that the user has not configured a contrast preference.

If the value is set to `more`, the user requests a higher level of contrast. Conversely, a value of `less` indicates that a lower contrast is preferred.

The value `custom` indicates that the user prefers a specific set of colors. This color palette is typically defined through the `forced-colors` media feature, which is explained in the following section. [4]

4.1.3. Forced Colors

The `forced-colors` media feature detects whether the user agent has enabled a forced color mode, such as Windows High Contrast. This media feature has the value `active` when such a mode is enabled and `none` when it is not.

When forced colors mode is active, many color values defined in the style sheet are overridden by the browser during the painting process. The exact colors applied depend on the semantic role of each element. In addition, browsers may draw backplates behind text to preserve legibility and maintain sufficient contrast when text appears on top of images or patterned backgrounds.

Developers are generally not encouraged to create a separate design specifically for users who enable forced colors mode. Instead, this media query should be used to make targeted adjustments when certain components do not work well in this context. For example, a button that relies solely on `box-shadow` to stand out from

the background may lose its visual distinction. In such cases, replacing the shadow with a visible border can provide a more robust solution. [5]

4.1.4. Reduced Transparency

The `prefers-reduced-transparency` media feature indicates whether a user has enabled a system setting to reduce transparent or translucent visual effects. If the value is set to `reduce`, transparency effects such as blurred overlays or frosted glass backgrounds should be minimized or removed. A value of `no-preference` indicates that the user has not expressed a preference and that the default design can be used.

Reducing transparency can improve contrast and readability, particularly for users who find translucent interface elements distracting or difficult to perceive.

Support for this media feature is currently limited and it is not yet fully supported by Firefox and Safari. [6]

4.1.5. Color Scheme

A more widely adopted media feature is `prefers-color-scheme`. It indicates whether the user prefers a light or dark color theme configured through the operating system or the user agent. The value `light` represents a light color theme, while `dark` represents a dark color theme. [7]

4.1.6. Inverted Colors

The `inverted-colors` media feature detects whether the user agent is displaying content with inverted colors. Color inversion can introduce visual side effects, such as shadows appearing as highlights, which may reduce the readability of the content. By responding to this media feature, developers can make targeted adjustments to preserve the visual integrity of the page.

Support for this media feature is currently limited and, at the time of writing, it is only supported by Safari. [8]

4.1.7. Accessing Media Features through JavaScript

The `window` interface provides the `matchMedia()` method, which returns a `MediaQueryList` object. This object can be used to determine whether the current `document` matches a given media query string. Listing 3 demonstrates this approach using the `prefers-color-scheme` media feature.

```
const isDarkTheme = window.matchMedia("(prefers-color-scheme:
dark)").matches;
```

Listing 3: Checking whether the user's system settings prefer a dark color scheme.

This allows developers to respond to user preferences not only in CSS, but also within JavaScript logic and application-specific features. [9]

4.1.8. Changes in the World of Media Queries

CSS media queries continue to evolve, but one important aspect of visual accessibility is still not addressed. Different forms of color blindness and other impairments that affect color perception are not yet covered by an official media feature.

An open issue in the W3C’s `csswg-drafts` repository proposes a new media feature called `color-vision-adjustment`. Its goal is to improve web accessibility by allowing developers to respond directly to the needs of users with color vision deficiencies.

The proposal includes support for several specific conditions, including `protanopia` (red deficiency), `deuteranopia` (red-green deficiency), `tritanopia` (blue-yellow deficiency), and `achromatopsia` (near-total color blindness, resulting in grayscale vision). These impairments were introduced in the previous P7 project through bookmarklets that simulated their visual effects.

A media feature of this kind would increase awareness of color vision deficiencies and provide developers with a standardized way to adapt interfaces to the needs of affected users. [10]

4.1.9. Deprecated Features

Earlier versions of CSS included media types such as `embossed`, `aural`, and `braille`, which were intended to address specific accessibility needs. These media types were later deprecated because the distinction between device categories was expected to become less meaningful over time. In addition, media types describe classes of devices rather than the actual capabilities they support. [11], [12]

4.1.10. Security Concerns

Information exposed through media queries can be used to build a browser fingerprint, which may allow a device to be identified and tracked across websites. To reduce this risk, browsers may deliberately modify or generalize certain reported values.

Some browsers also provide settings that further limit this information. For example, Firefox offers the “Resist Fingerprinting” option, which causes many media queries to return standardized values instead of device-specific settings. [12]

4.2. CSS Custom Properties

CSS custom properties, also known as CSS variables, are named values that can be reused throughout a style sheet. Their value depends on the scope in which they are defined, but within that scope they evaluate to the same value wherever they are used. This makes styles easier to maintain and reduces duplication.

Custom properties are defined using a name that begins with two dashes, `--`, followed by a descriptive identifier. Their value can be accessed using the CSS `var()` function. Listing 4 demonstrates how the custom property `--primary-color` can be defined once and reused throughout a website. This approach is particularly useful when implementing multiple themes, such as light and dark mode, because colors can be changed in a single central location. [13]

```
:root {
  --primary-color: #204CCF;
  --primary-color-contrast: #FFFFFF;
}

button.primary {
  background-color: var(--primary-color);
  color: var(--primary-color-contrast);
}

h1, h2, h3 {
  color: var(--primary-color);
}
```

Listing 4: Defining a primary color custom property that can be reused throughout the website.

The `@property` at-rule allows custom properties to be defined more precisely. It makes it possible to specify the expected value type using `syntax`, whether the property is inherited by default using `inherits`, and an initial value using `initial-value`. Possible syntax definitions include `<color>`, `<number>`, and `<url>`. Listing 5 shows how the previous `--primary-color` example can be expressed using this at-rule. [14], [15]

```
@property --primary-color {
  syntax: "<color>";
  inherits: false;
  initial-value: #204CCF;
}
```

Listing 5: Defining a primary color custom property using the `@property` at-rule.

As shown in Listing 6, custom properties can also be defined in JavaScript using `registerProperty()` or using `setProperty()`. [16], [17]

```
window.CSS.registerProperty({
  name: '--primary-color',
  syntax: '<color>',
  inherits: false,
  initialValue: '#204CCF ',
});

const root = document.documentElement;
root.style.setProperty('--primary-color-contrast', '#FFFFFF');
```

Listing 6: Using `registerProperty()` and `setProperty()` in JavaScript to define CSS custom properties.

Custom properties can also be read in JavaScript using `getPropertyValue()`, as shown in Listing 7. This makes it possible to react to dynamically changed values and use them in application logic.

```
const root = document.documentElement;
const contrastColor = getComputedStyle(root).getPropertyValue('--contrast-color');
```

Listing 7: Reading a CSS custom property in JavaScript using `getPropertyValue()`.

4.2.1. Global State Management

CSS custom properties are not limited to storing design values such as colors. They can also be used to manage application state. Because custom properties can be read and modified in both CSS and JavaScript, they provide a convenient way to store configuration values that directly influence the user interface.

For example, a user may not have enabled reduced motion at the operating system or browser level, but may choose to disable animations through a website-specific setting. This preference can be stored in a custom property such as

`--prefers-reduced-motion: true`. The `@container` at-rule can then react to this value and apply different styles at runtime, as shown in Listing 8. [18]

```
:root {
  --prefers-reduced-motion: false;
}

@container style(--prefers-reduced-motion: true) {
  .animated-content {
    animation: none !important;
  }
}
```

Listing 8: Using the CSS `@container` at-rule to disable animations based on a custom property.

Another approach is to use a CSS attribute selector to override custom properties when a different color theme is selected. For example, the operating system and browser may indicate a preference for a light theme, while the user explicitly selects a dark theme on the website. In this case, a custom property such as

`--prefers-dark-theme: true` can be used to switch the application’s color palette, as demonstrated in Listing 9. [19]

```

:root {
  --prefers-dark-theme: false;

  --text-color:      #222222;
  --bg-color:        #FCFCFC;
  --border-color:    #EEEEEE;
  --primary-accent:  #0056B3;
}

:root[style*="--prefers-dark-theme: true"] {
  --text-color:      #EDED;
  --bg-color:        #212121;
  --border-color:    #232323;
  --primary-accent:  #6DB3F4;
}

```

Listing 9: Using a CSS attribute selector to switch to a dark color theme based on a custom property.

4.2.2. Custom Property Toggle

In modern CSS, developers often face the challenge of managing repetitive declarations, especially when implementing light and dark themes. Whether the switch is handled through classes or through

`@media (prefers-color-scheme: dark)`, the same property names typically need to be declared multiple times with different values. A lesser-known detail in the CSS specification makes it possible to implement this kind of state management more elegantly by using custom properties as toggle values.

According to the CSS specification, a custom property must contain at least one token to be considered valid. A single whitespace character satisfies this requirement and can therefore be used to simulate boolean-like values, as shown in Listing 10. [20]

```

:root {
  --true: ;
  --false: initial;
}

```

Listing 10: Both `' '` and `initial` are valid values for custom properties.

In traditional theming setups, each variable must be defined separately for both the light and dark theme. By using the pseudo-boolean values introduced in Listing 10, the theme state can be stored in a single custom property, as demonstrated in Listing 11.

```

:root {
  --is-light-theme: var(--true);
}

@media (prefers-color-scheme: dark) {
  :root {
    --is-light-theme: var(--false);
  }
}

```

Listing 11: Assigning either `' '` or `initial` depending on the active theme creates the basis for the toggle logic.

When `--is-light-theme` is used as a prefix in another custom property declaration, the resulting declaration is either valid or invalid depending on its value. If the prefix expands to a whitespace character, the declaration remains valid. If it expands to `initial`, the declaration becomes invalid. In that case, the fallback value provided to `var()` is used automatically, as illustrated in Listing 12.

```

:root {
  --color-text--light: var(--is-light-theme) black;
  --color-text--dark: white;

  --color-background--light: var(--is-light-theme) white;
  --color-background--dark: black;

  --color-text: var(--color-text--light,
                    var(--color-text--dark));
  --color-background: var(--color-background--light,
                           var(--color-background--dark));
}

```

Listing 12: When a custom property becomes invalid, `var()` automatically uses the fallback value.

Multiple fallback values based on different toggle properties can be chained using the CSS `var()` function. This enables centralized state management, where the custom properties used throughout the entire style sheet are controlled from a single location. [21], [22]

An example of this theme toggle is available in the project's repository: https://github.com/Flaverus/P8_sandbox/tree/main/examples/theme-switch

4.3. CSS Functions

CSS provides a wide range of built-in functions that simplify common styling tasks. Some of these functions are particularly useful for accessibility. One notable example is `contrast-color()`, which became broadly available during the implementation of this project in April 2026 and is supported by current browser versions. The function returns either black or white, depending on which of the two provides the higher contrast against the color passed as its argument. Listing 13 shows this approach in the context of a `button` element. [23]

```
button {  
  background-color: var(--button-color);  
  color: contrast-color(var(--button-color));  
}
```

Listing 13: Setting the text color of a button to black or white based on the background color.

Using either black or white for text is a reliable way to ensure strong contrast. Depending on the background color, however, the result may appear visually harsh and less pleasant to read. Another recently introduced CSS function is `light-dark()`, which accepts two colors or images. The first value is used when a light color scheme is active, while the second value is used for a dark color scheme. Listing 14 demonstrates this approach using custom properties. [24]

```
body {  
  color: light-dark(var(--color-text--light),  
                   var(--color-text--dark));  
  background-color: light-dark(var(--color-background--light),  
                              var(--color-background--dark));  
}
```

Listing 14: Defining text and background colors based on the active color theme using custom properties.

4.3.1. CSS Custom Functions

Although the growing set of built-in CSS functions covers many use cases, some scenarios still require functionality that cannot be expressed with the currently available primitives. In such cases, developers may need to combine multiple functions or implement custom logic. This is where the experimental `@function` at-rule becomes relevant.

This feature is conceptually similar to CSS custom properties, as custom functions also use names that begin with `--`. A function is declared using the `@function` keyword, followed by a custom name and an optional list of parameters. Both the function name and its parameters start with `--` and are followed by case-sensitive identifiers defined by the developer. Inside the function body, the expression assigned to `result:` is evaluated and returned, as shown in Listing 15. [25]

```
@function --color-contrast(--color <color>) returns <color> {  
  result: oklch(from var(--color) calc((0.5 - 1) * infinity) 0 0);  
}
```

Listing 15: A custom function that provides behavior similar to `contrast-color()` and was created before that function became widely available.

As with CSS custom properties, CSS data types such as `<color>`, `<number>`, and `<string>` can be specified for both function parameters and the return value.

4.4. Color Spaces

There are many different models for describing the colors perceptible to the human eye. Each model has its own strengths and weaknesses and is used in different contexts, ranging from print and photography to digital displays.

For many years, CSS relied primarily on the RGB color space, which limited how colors could be described and manipulated. The CSS Color Module Level 4 specification introduced support for additional color spaces and more advanced color functions, significantly expanding the possibilities for working with color in CSS. Development in this area continues, and the W3C is currently working on the CSS Color Module Level 5 specification. [26]

4.4.1. RGB

The RGB color model represents colors by combining different intensities of the primary colors `red`, `green`, and `blue`. An optional alpha channel can be added to define the opacity of the resulting color.

RGB has been supported in CSS since its early days and can be expressed using the `rgb()` function or hexadecimal notation. For example, the color shown in Listing 16 can also be written as `#2d1fb180`. [26]

```
rgb(45 31 177 / 0.5);
```



Listing 16: Using `rgb()` to define a color from the Kolibri palette with 50% opacity.

4.4.2. HSL

CSS Color Module Level 3 introduced the HSL color model to CSS. HSL describes colors using three components: `hue`, `saturation`, and `lightness`, as illustrated in Listing 17. The hue is represented as an angle on the color wheel, while saturation and lightness define the intensity and brightness of the color. As with RGB, an optional alpha channel can be added to control opacity.

HSL makes it easier to create variations of a color because saturation and lightness can be adjusted directly without recalculating individual RGB values. This is particularly useful when generating lighter or darker shades of the same base color. [26]

```
hsl(256 82 55 / 0.6);
```



Listing 17: Using `hsl()` to define a color from the Kolibri palette with 60% opacity.

4.4.3. CIELAB Colors

Perceptual color spaces such as Oklab and OkLCH make it possible to define colors in a way that more closely matches human vision. They also provide access to a wider range of colors than those that can be represented in the traditional sRGB color space.

The `oklab()` function describes a color using three components: `lightness`, the `a` axis representing red-green variation, and the `b` axis representing yellow-blue variation. The related `oklch()` function uses `lightness`, `chroma`, and `hue` instead. Both functions support an optional alpha channel to control opacity.

These color models are widely regarded as the current state of the art for working with color in CSS because they produce more perceptually uniform results. In practice, this means that adjusting lightness or chroma leads to more predictable visual changes than with older color models. [26]

```
oklab(0.638 0.176 -0.279 / 0.7);  
oklch(0.718 0.255 301.5 / 0.7);
```



Listing 18: Using `oklab()` and `oklch()` to define a color from the Kolibri palette with 70% opacity.

4.4.4. HWB

The `hwb()` color function was introduced alongside the newer CSS color features. It describes colors using three components within the RGB color space: `hue`, `whiteness`, and `blackness`. In practice, it offers an alternative to `hsl()` that many developers find more intuitive when adjusting tints and shades. An example is shown in Listing 19. [26]

```
hwb(325 18 0 / 0.3);
```



Listing 19: Using `hwb()` to define a color from the Kolibri palette with 30% opacity.

4.5. Contrast Perception

Contrast is essential for distinguishing elements and understanding their relationships. Differences in brightness, size, texture, and shape all contribute to visual contrast. On the web, variations in lightness are the primary means of ensuring text readability and clearly separating interface elements.

The following section introduces the contrast requirements defined in WCAG 2.x and provides an overview of the emerging approach proposed for WCAG 3.0.

4.5.1. Relative Luminance

The contrast requirements in WCAG 2.x are based on the relative luminance of text and its background, independent of hue. This approach assumes that hue and saturation have little influence on reading performance, even for users with color vision deficiencies. Since the inability to distinguish certain colors does not usually affect the perception of light and dark, color itself is not treated as a primary factor in the contrast calculation.

In practice, the perceived contrast may differ from the theoretical value. Smaller or thinner fonts can appear noticeably lighter than the color specified in CSS due to anti-aliasing and font rendering. As a result, text may appear to have lower contrast than the calculated ratio suggests.

WCAG 2.x requires a minimum contrast ratio of 4.5:1 for normal text to meet Level AA. Large text must reach at least 3:1. For Level AAA, the minimum contrast ratio increases to 7:1. These thresholds are based in part on recommendations from ISO 9241-3 and are intended to compensate for reduced contrast sensitivity in users with moderate visual impairments. Users with more severe vision loss typically rely on assistive technologies such as screen magnifiers or screen readers.

Color vision deficiencies vary significantly, making it difficult to define universally effective color combinations based solely on hue. This is one of the main reasons why WCAG evaluates contrast primarily through relative luminance.

One notable exception is **protanopia**, in which red tones may appear significantly darker than expected. As a result, red text on dark backgrounds can provide much less contrast than the calculated ratio suggests and should generally be avoided. [27]

The formula shown in Listing 20 is used by WCAG 2.x to calculate relative luminance, which serves as the basis for determining contrast ratios. [28]

$$L = 0.2126 \cdot R + 0.7152 \cdot G + 0.0722 \cdot B$$

$$\text{Where } C = \begin{cases} \frac{C_{sRGB}}{12.92} & \text{if } C_{sRGB} \leq 0.03928 \\ \left(\frac{C_{sRGB} + 0.055}{1.055} \right)^{2.4} & \text{otherwise} \end{cases}$$

Example: Dodger Blue `rgb(30, 144, 255)`

1. Normalize (/255)

$$R_{sRGB} = \frac{30}{255} = 0.1176$$

$$G_{sRGB} = \frac{144}{255} = 0.5647$$

$$B_{sRGB} = \frac{255}{255} = 1.0000$$

2. Linearize

$$R = 0.0123$$

$$G = 0.2747$$

$$B = 1.0000$$

3. Calculate Luminance (L)

$$L = (0.2126 \cdot 0.0123) + (0.7152 \cdot 0.2747) + (0.0722 \cdot 1.0000) = \mathbf{0.2712}$$

Calculate Contrast Ratio

$$\text{Ratio} = \frac{L1 + 0.05}{L2 + 0.05}$$

Listing 20: Step-by-step calculation of relative luminance based on the W3C sRGB formula. [28]

4.5.2. Perceptual Color Models

The contrast model used in WCAG 2.x is based on relative luminance. In the future, it is expected to be replaced by the Accessible Perceptual Contrast Algorithm (APCA), which forms part of the ongoing WCAG 3.0 work. Advances in display technology, modern web design, and vision science have shown that the approach introduced nearly two decades ago in WCAG 2.x no longer reflects how contrast is perceived in practice.

WCAG 2.x follows a binary model in which a contrast ratio either passes or fails. APCA takes a different approach by accounting for the non-linear nature of human perception and by evaluating contrast in the context in which the colors are used.

Readability depends on more than the luminance difference between two colors. Font size, font weight, stroke thickness, and surrounding context all influence how contrast is perceived. High light-dark contrast generally improves readability, but smaller and thinner text requires a greater difference in lightness to remain legible, as illustrated in Figure 1.

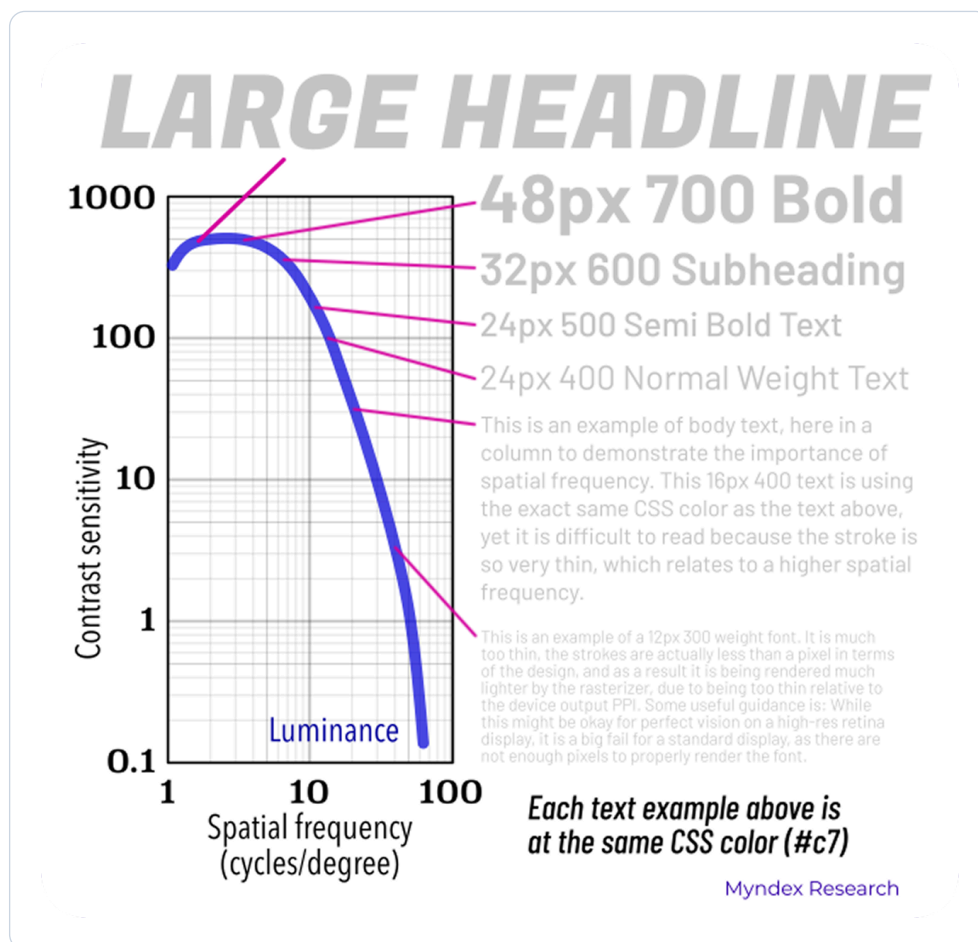


Figure 1: This chart illustrates the spatial dependence of human contrast sensitivity using text samples. [29]

Non-text elements such as icons typically require less contrast than body text. More generally, the appropriate contrast between two colors depends on the specific use case, including the size, thickness, and purpose of the element.

The contrast formula used in WCAG 2.x has been criticized for many years. Case studies comparing white and black text on colored backgrounds have shown that

combinations classified as inaccessible under WCAG 2.x were sometimes perceived as more readable by participants with color vision deficiencies. [30]

APCA introduces a new metric called lightness contrast, expressed as L^c . Instead of assigning a universal pass-or-fail threshold, it estimates readability based on the perceptual relationship between foreground and background colors within their actual visual context. [29]

4.6. Canvas

The HTML `<canvas>` element is used to draw graphics and animations directly within a web page. By default, the element has no visual representation of its own and must be rendered through JavaScript using either the `Canvas API` or the `WebGL API`.

The `Canvas API` is primarily designed for two-dimensional graphics and is commonly used for tasks such as game rendering, data visualization, image manipulation, and real-time video processing. The `WebGL API` extends these capabilities by providing hardware-accelerated rendering for both two-dimensional and three-dimensional graphics.

In addition to drawing content, the canvas also allows pixel data to be read and analyzed. The `ImageData` interface provides access to the underlying RGBA values of a rendered image. Listing 21 demonstrates how the color values of a selected area can be extracted in JavaScript. [31], [32], [33]

```
const canvas = document.createElement('canvas');
const ctx    = canvas.getContext('2d');

ctx.fillStyle = 'oklab(0.638 0.176 -0.279)';
ctx.fillRect(0, 0, 100, 100);

const red    = ctx.getImageData(10, 10, 10, 10).data[0];
const green  = ctx.getImageData(10, 10, 10, 10).data[1];
const blue   = ctx.getImageData(10, 10, 10, 10).data[2];
const alpha  = ctx.getImageData(10, 10, 10, 10).data[3];
```

Listing 21: Creating a purple 100×100 pixel square and extracting the RGBA values from a 10×10 pixel region starting at coordinates $x = 10$ and $y = 10$.

4.7. Ishihara

The Ishihara color test, named after the Japanese ophthalmologist Shinobu Ishihara, was developed in 1917 to detect red-green color vision deficiencies. The test consists of a series of circular plates composed of pseudo-isochromatic dots. These dots vary in size and color and are arranged in patterns that appear distinct to individuals with typical color vision, while blending together for people with certain forms of color blindness. [34]

An example of such a plate is shown in Figure 2. It uses orange and teal tones and is designed to remain visible regardless of the viewer’s color perception. Plates of this kind are commonly used as introductory demonstration images at the beginning of an Ishihara test.

4.7.1. Functional Application in Contrast Testing

Although originally developed as a diagnostic tool, the underlying principle of Ishihara plates also provides a useful framework for evaluating color contrast in interface design.

By placing many small dots of different colors next to each other, the method forces the visual system to rely primarily on chromatic contrast to identify a pattern. In a custom accessibility tool, if the foreground and background colors visually blend together within such a plate, this indicates that the chosen color combination may not provide sufficient perceptual contrast.

Custom plates can be generated using a specific brand palette to verify that key colors remain distinguishable not only for users with typical color vision, but also for users with conditions such as `protanopia` or `deuteranopia`.

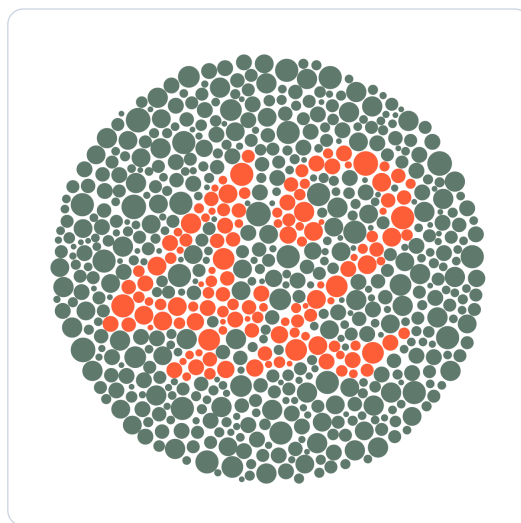


Figure 2: A high-contrast demonstration plate designed to remain legible across all common vision types.

5. Methodology

6. Implementation

This chapter contains suggestions that extend browser standards regarding improved accessibility. The **preferences widget** allows user-specific configuration of an individual website, the **contrast color function** is a function that extends the new `contrast-color()` CSS function by adding a color component, and the **interactive ishihara plate** allows users to manually check color contrasts, as well as test color values and their contrasts using different color filters that simulate various forms of color blindness.

6.1. Preferences Weidget

CSS media queries already allow to address certain user needs. However, this requires that the user has configured specific needs in their operating system or browser settings. If no settings have been configured, there is no way to take the necessary precautions to offer the user a tailored accessibility experience. If incorrect settings exist, or if certain measures are not desired by the user in specific scenarios, these would have to be adjusted globally, even if the circumstances differ only for a single website. Furthermore, there are accessibility aspects that are not yet covered by CSS media queries and therefore cannot be directly considered.

To prevent these limitations and give the user complete freedom, this preferences widget, shown in Figure 3, was created. It refers to existing system settings but allows users to customize them and offers the possibility to extend the limited scope of CSS media queries to suit specific websites.

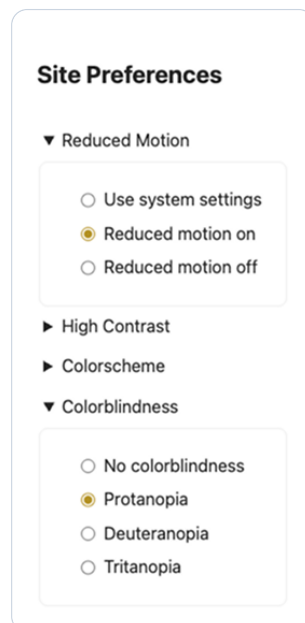


Figure 3: A screenshot of the preferences widget where custom settings for reduced motion and colorblindness on a pagespecific level were configured.

6.1.1. JavaScript Part

Existing media queries are used as a basis, provided the relevant setting is included in the scope of CSS media queries. It is possible to configure settings specifically based on the operating system or browser settings. The system settings are used as the initial value when a page is visited for the first time. All values are stored in the browsers `localStorage`, ensuring that settings are not lost on page refreshes and are consistently available across a web application. The values are set as CSS custom properties on the pseudo-class `:root`, from where they can then be used, as shown in Listing 22.

```
const setAccessibilityProperty = (property,
                                value,
                                mediaQueryString) => {

  localStorage.setItem(property, value);

  if(value === 'system' && mediaQueryString !== '') {
    const systemSetting = window.matchMedia(mediaQueryString).matches;
    root.style.setProperty(property, systemSetting);
  } else {
    root.style.setProperty(property, value);
  }
}
```

Listing 22: The `setAccessibilityProperty()` saves a property value to the `localStorage` and creates a custom property on `:root`.

With the function `getAccessibilityProperty()` shown in Listing 23 previously saved configurations are read from the `localStorage` or set to the system default if they have not been defined yet.

```
const getAccessibilityProperty = (property) => {
  const savedProperty = localStorage.getItem(property);

  if(savedProperty) {
    return savedProperty;
  } else {
    return 'system';
  }
}
```

Listing 23: With the usage of `getAccessibilityProperty()` previously saved values are returned or the systems default value will be used.

The widget makes use of both the `setAccessibilityProperty()` and `getAccessibilityProperty()` functions to set the custom properties on page load as it can be seen in Listing 24. The first parameter in `setAccessibilityProperty()` is the custom property name that will be set to the `:root`, the second parameter contains the return value of the `getAccessibilityProperty()` function which is either the previously saved value from the `localStorage` or the string 'system' as default value if no saved data exists so far, as well as the CSS media query string as the third and last parameter if only a `true` or `false` value is considered for this property.

```
setAccessibilityProperty('--prefers-reduced-motion',
  getAccessibilityProperty(
    '--prefers-reduced-motion'),
  '(prefers-reduced-motion: reduce)');
setAccessibilityProperty('--prefers-contrast',
  getAccessibilityProperty(
    '--prefers-contrast'),
  '(prefers-contrast: more)');
setAccessibilityProperty('--prefers-dark-theme',
  getAccessibilityProperty(
    '--prefers-dark-theme'),
  '(prefers-color-scheme: dark)');
setAccessibilityProperty('--prefers-colorblind-mode',
  getAccessibilityProperty(
    '--prefers-colorblind-mode'),
  '');
```

Listing 24: Setting the global custom propperties on page load with the properties custom name, the return value of `getAccessibilityProperty()` to consider already set values and system settings, as well as the media query in question.

As a next step the function `syncWidgetOption()` seen in Listing 25 synchronizes the values from the CSS custom properties to the radio buttons representing the settings in the UI. This makes use of the custom properties on the `:root` as single source of truth to not having to propagate data change to different areas of the code as it is accessible both from CSS and JavaScript.

```
const syncWidgetOption = (option, property) => {
  const value = root.style.getPropertyValue(property);

  option.forEach((radio) => {
    radio.checked = radio.value === value;
  });
}

syncWidgetOption(motion, '--prefers-reduced-motion');
syncWidgetOption(contrast, '--prefers-contrast');
syncWidgetOption(colorscheme, '--prefers-dark-theme');
syncWidgetOption(colorblindness, '--prefers-colorblind-mode');
```

Listing 25: Setting the global custom properties on page load with the properties custom name, the return value of `getAccessibilityProperty()` to consider already set values and system settings, as well as the media query in question.

Lastly each setting needs to be updated in both the `localStorage` as well as the property value in the document if the selection in the radio button group changes which is shown in Listing 26.

```

addOptionEventListener(motion,
    '--prefers-reduced-motion',
    '(prefers-reduced-motion: reduce)');
addOptionEventListener(contrast,
    '--prefers-contrast',
    '(prefers-contrast: more)');
addOptionEventListener(colorscheme,
    '--prefers-dark-theme',
    '(prefers-color-scheme: dark)');
addOptionEventListener(colorblindness,
    '--prefers-colorblind-mode',
    '');

const addOptionEventListener = (option, property, mediaQueryString) =>
{
    option.forEach((radio) => {
        radio.addEventListener('change', () => {
            if(radio.checked) {
                setAccessibilityProperty(property, radio.value,
mediaQueryString);
            }
        });
    });
}

```

Listing 26: Reacting to changes in the radio button group to update the users preference setting.

6.1.2. CSS Part

With the CSS custom properties that represent the users preferences in place the next step is to make use of these properties and build different representations of a website for the possible combinations of these configurations. In Listing 27 custom CSS properties are defined, holding the base style for a website and with the CSS attribute selector `[]` these base styles are overwritten for the dark color theme if the user prefers that one.

```

:root {
  --bg-color:      #fcfcfc;
  --text-color:    #222222;
  --content-bg:    #ffffff;
  --border-color:  #eeeeee;
  --primary-accent: #0056b3;
  --shadow:        0 2px 8px rgba(0, 0, 0, 0.1);

  color-scheme: light;
}

:root[style*="--prefers-dark-theme: true"] {
  --bg-color:      #212121;
  --text-color:    #EDED;
  --content-bg:    #2E2E2E;
  --border-color:  #232323;
  --primary-accent: #6DB3F4;
  --shadow:        0 2px 8px rgba(0, 0, 0, 0.5);

  color-scheme: dark;
}

```

Listing 27: Overwriting the base styling CSS custom properties with the CSS attribute selector on `:root` for a dark color scheme styling.

The more configuration properties available the more combinations of different settings must be considered. Listing 28 shows how the base styling properties are set to yet other values if both the color scheme is set to dark as well as the high contrast styling is enabled, resulting in another visual representation of the website.

```

:root[style*="--prefers-dark-theme: true"]
[style*="--prefers-contrast: true"] {
  --bg-color:      #000000;
  --text-color:    #ffffff;
  --content-bg:    #000000;
  --border-color:  #ffffff;
  --primary-accent: #80c5ff;
  --shadow:        none;
}

```

Listing 28: Considering both the color scheme as well as the high contrast setting leads to another styling.

If not only the base styles change based on the widget configuration but also additional rules must be applied the `@container` CSS rule with the `style()` query provide the needed functionality. This allows to select the root level for styling as the properties are defined on `:root` and therefore can be applied throughout the whole application. Listing 29 considers disabling animations and transitions when the user prefers reduced motion.

```
@container style(--prefers-reduced-motion: true) {  
  .animated-contents {  
    animation: none !important;  
  }  
  
  .transitional-contents {  
    transition: none !important;  
  }  
}
```

Listing 29: Additional styling based on the custom properties can be added with the `@container` rule in combination with the `style()` query.

6.2. Contrast Color Functions

In the theory chapter the `contrast-color()` CSS function was used as an example and how such a function could be built with CSS custom functions. As mentioned before, this CSS function returns for a given color either white or black, depending on which of those two have a higher lightness contrast to the given color. It was also mentioned that having pure black or white can feel harsh to read.

To solve this problem an improved version of the `contrast-color()` that can be inspected in Listing 30 has been created during this project. This custom function takes a parameter from type `<color>` as well as an optional `<percentage>` parameter. In a first step it is calculated whether black or white has a better contrast to the given color with the help of `oklch()`. If a color has a lightness value of 50% or above the resulting color for `--black-or-white` is black and otherwise white. In a second step the resulting black or white is put into the `color-mix()` function that allows to mix two colors together. Here the `--intensity` parameter comes into play, as the defined percentage defines, how much from the given color should be mixed back into the resulting contrast color to smoothen it out further.

To not have the problem of a contrast that hurts your eyes when reading in the first place, the lightness of `--black-or-white` is reduced with the usage of the `clamp()` function. This part ensures that the lightness for a light contrast color is at least 15% and a dark contrast color will not exceed 97.5%, smoothening the resulting color even if there is no color from the selected color mixed back in.

```
@function --color-contrast(--color <color>, --intensity <percentage>:
0%) returns <color> {
  --black-or-white: oklch(from var(--color) calc((0.5 - 1) * infinity)
0 0);

  result: color-mix(in oklch, oklch(from var(--black-or-white)
clamp(0.15, 1, 0.975) c h), var(--color) var(--intensity));
}
```

Listing 30: A custom color contrast function that adjusts the lightness of the resulting black or white and allows to mix back in some parts of the original color.

The example page from the examples collection for this custom contrast color function also calculates the resulting contrast ratio as defined for WCAG 2.x as it can be seen in Figure 4 below.

Black or White as Contrast with Optional Color Saturation

Pick a color and get either black or white as contrast color, depending on the color's luminance. Additionally, a saturation percentage can be defined to move from a sharp contrast to a more pleasant one with color added.

Configuration

Background color

Saturation percentage

Results

Background: #be58fd

Foreground: oklch(0.200943 0.024045 354.977)

Contrast Ratio: 5.19:1

Copy Resulting Color





Figure 4: A screenshot of the configuration UI for the custom contrast color function.

To be able to calculate the relative contrast ratio of two colors the `RGB` values of each color are needed. When working with e.g. `oklch()` these values are not directly accessible within the browser as the computed styles are in the `oklch()` format too. A workaround to extract the needed values that was used for this example is to use the HTML `<canvas>` element. This element allows styling with the common CSS color functions and allows to extract `RGBA` values through the `getImageData()` function. With this little trick shown in Listing 31 it is possible to extract these values from every color, no matter the original format.

```
const parseColorToRGBA = (colorString) => {
  const canvas = document.createElement('canvas');
  const ctx    = canvas.getContext('2d');

  ctx.fillStyle = colorString;
  ctx.fillRect(0, 0, 1, 1);

  // Returning data array containing RGBA values as follows: r, g, b, a
  return ctx.getImageData(0, 0, 1, 1).data;
};
```

Listing 31: Extracting the `RGBA` values from any color in any format through the `<canvas>` element.

6.3. Interactive Ishihara Plate

7. Discussion

8. Conclusions

Bibliography

- [1] Florian Schnidrig, “Accessibility on the Web – Where We Stand.” Accessed: Apr. 13, 2026. [Online]. Available: <https://accessible-web-initiative.gitbook.io/accessibility-on-the-web-where-we-stand>
- [2] Mozilla Corporation, “Using media queries.” Accessed: Mar. 09, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Media_queries/Using
- [3] Mozilla Corporation, “prefers-reduced-motion.” Accessed: Apr. 16, 2026. [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/At-rules/@media/prefers-reduced-motion>
- [4] Mozilla Corporation, “prefers-contrast.” Accessed: Apr. 16, 2026. [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/At-rules/@media/prefers-contrast>
- [5] Mozilla Corporation, “forced-colors.” Accessed: Mar. 09, 2026. [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/At-rules/@media/forced-colors>
- [6] Mozilla Corporation, “prefers-reduced-transparency CSS media feature.” Accessed: Apr. 23, 2026. [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/At-rules/@media/prefers-reduced-transparency>
- [7] Mozilla Corporation, “prefers-color-scheme.” Accessed: Apr. 16, 2026. [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/At-rules/@media/prefers-color-scheme>
- [8] Mozilla Corporation, “inverted-colors.” Accessed: Mar. 09, 2026. [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/At-rules/@media/prefers-color-scheme>
- [9] Mozilla Corporation, “Window: matchMedia() method.” Accessed: Apr. 16, 2026. [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/API/Window/matchMedia>
- [10] Naftali Peles, “[mediaqueries] CSS Media Feature for Color Vision Adjustments.” Accessed: Mar. 16, 2026. [Online]. Available: <https://github.com/w3c/csswg-drafts/issues/10335>
- [11] World Wide Web Consortium, “Media Queries – disposition of comments.” Accessed: Apr. 16, 2026. [Online]. Available: <https://www.w3.org/Style/css3-updates/css3-mediaqueries-comments>
- [12] Mozilla Corporation, “@media.” Accessed: Apr. 16, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/At-rules/@media#media_types

- [13] Mozilla Corporation, “Using CSS custom properties (variables).” Accessed: Apr. 23, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Cascading_variables/Using_custom_properties
- [14] Mozilla Corporation, “@property CSS at-rule.” Accessed: Apr. 23, 2026. [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/At-rules/@property>
- [15] Mozilla Corporation, “syntax CSS at-rule descriptor.” Accessed: Apr. 23, 2026. [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/At-rules/@property/syntax>
- [16] Mozilla Corporation, “CSS: registerProperty() static method.” Accessed: Apr. 23, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/CSS/registerProperty_static
- [17] Mozilla Corporation, “CSSStyleDeclaration: setProperty() method.” Accessed: Apr. 23, 2026. [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/API/CSSStyleDeclaration/setProperty>
- [18] Mozilla Corporation, “@container CSS at-rule.” Accessed: Apr. 23, 2026. [Online]. Available: <https://developer.mozilla.org/de/docs/Web/CSS/Reference/At-rules/@container>
- [19] Mozilla Corporation, “Attribute selectors.” Accessed: Apr. 23, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Selectors/Attribute_selectors
- [20] World Wide Web Consortium, “CSS Custom Properties for Cascading Variables Module Level 1.” [Online]. Available: <https://www.w3.org/TR/css-variables-1/#syntax>
- [21] Fynn Ellie Becker, “CSS Custom Property toggles for themes.” [Online]. Available: <https://fynn.be/blog/css-custom-property-toggles-themes/>
- [22] Mozilla Corporation, “var() CSS function.” Accessed: Apr. 29, 2026. [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Values/var>
- [23] Mozilla Corporation, “contrast-color() CSS function.” Accessed: May 04, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Values/color_value/contrast-color
- [24] Mozilla Corporation, “light-dark() CSS function.” Accessed: May 04, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Values/color_value/light-dark

- [25] Mozilla Corporation, “@function CSS at-rule.” Accessed: May 04, 2026. [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/At-rules/@function>
- [26] Brian Smith, “New functions, gradients, and hues in CSS colors (Level 4).” Accessed: Apr. 23, 2026. [Online]. Available: <https://developer.mozilla.org/en-US/blog/css-color-module-level-4/>
- [27] World Wide Web Consortium, “Contrast (Minimum) (Level AA).” Accessed: Apr. 20, 2026. [Online]. Available: <https://www.w3.org/WAI/WCAG22/Understanding/contrast-minimum.html>
- [28] World Wide Web Consortium, “Relative luminance.” Accessed: Apr. 20, 2026. [Online]. Available: https://www.w3.org/WAI/GL/wiki/Relative_luminance
- [29] Andrew Somers, “Why APCA as a New Contrast Method?.” [Online]. Available: <https://git.apcacontrast.com/documentation/WhyAPCA.html>
- [30] Ericka O'Connor, “Orange You Accessible? A Mini Case Study on Color Ratio.” [Online]. Available: <https://www.bounteous.com/insights/2019/03/22/orange-you-accessible-mini-case-study-color-ratio/>
- [31] Mozilla Corporation, “<canvas> HTML graphics canvas element.” Accessed: May 11, 2026. [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/canvas>
- [32] Mozilla Corporation, “Canvas API.” Accessed: May 11, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Canvas_API
- [33] Mozilla Corporation, “ImageData.” Accessed: May 11, 2026. [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/API/ImageData>
- [34] Geoffrey Potjewyd, “The Color Code.” [Online]. Available: <https://theophthalmologist.com/issues/2022/articles/sep/the-color-code>

List of Figures

Figure 1	This chart illustrates the spatial dependence of human contrast sensitivity using text samples. [29]	19
Figure 2	A high-contrast demonstration plate designed to remain legible across all common vision types.	22
Figure 3	A screenshot of the preferences widget where custom settings for reduced motion and colorblindness on a pagespecific level were configured.	24
Figure 4	A screenshot of the configuration UI for the custom contrast color function.	32

List of Listings

Listing 1	Syntax of a media query, where <code><media-query-list></code> contains <code><media-type></code> values, <code><media-feature></code> values, and logical operators. . .	3
Listing 2	Example of a media query that changes the background color on smaller screens using the <code>screen</code> media type and the <code>max-width</code> media feature. .	4
Listing 3	Checking whether the user’s system settings prefer a dark color scheme. .	5
Listing 4	Defining a primary color custom property that can be reused throughout the website.	7
Listing 5	Defining a primary color custom property using the <code>@property</code> at-rule. .	8
Listing 6	Using <code>registerProperty()</code> and <code>setProperty()</code> in JavaScript to define CSS custom properties.	8
Listing 7	Reading a CSS custom property in JavaScript using <code>getPropertyValue()</code>	8
Listing 8	Using the CSS <code>@container</code> at-rule to disable animations based on a custom property.	9
Listing 9	Using a CSS attribute selector to switch to a dark color theme based on a custom property.	10
Listing 10	Both <code>'</code> and <code>initial</code> are valid values for custom properties.	10
Listing 11	Assigning either <code>'</code> or <code>initial</code> depending on the active theme creates the basis for the toggle logic.	11
Listing 12	When a custom property becomes invalid, <code>var()</code> automatically uses the fallback value.	11
Listing 13	Setting the text color of a button to black or white based on the background color.	12
Listing 14	Defining text and background colors based on the active color theme using custom properties.	12
Listing 15	A custom function that provides behavior similar to <code>contrast-color()</code> and was created before that function became widely available.	13
Listing 16	Using <code>rgb()</code> to define a color from the Kolibri palette with 50% opacity.	14
Listing 17	Using <code>hsl()</code> to define a color from the Kolibri palette with 60% opacity.	15
Listing 18	Using <code>oklab()</code> and <code>oklch()</code> to define a color from the Kolibri palette with 70% opacity.	15
Listing 19	Using <code>hwb()</code> to define a color from the Kolibri palette with 30% opacity.	16
Listing 20	Step-by-step calculation of relative luminance based on the W3C sRGB formula. [28]	18
Listing 21	Creating a purple 100 × 100 pixel square and extracting the RGBA values from a 10 × 10 pixel region starting at coordinates x = 10 and y = 10. .	21
Listing 22	The <code>setAccessibilityProperty()</code> saves a property value to the <code>localStorage</code> and creates a custom property on <code>:root</code>	25

Listing 23	With the usage of <code>getAccessibilityProperty()</code> previously saved values are returned or the systems default value will be used.	26
Listing 24	Setting the global custom propperties on page load with the properties custom name, the return value of <code>getAccessibilityProperty()</code> to consider already set values and system settings, as well as the media query in question.	26
Listing 25	Setting the global custom propperties on page load with the properties custom name, the return value of <code>getAccessibilityProperty()</code> to consider already set values and system settings, as well as the media query in question.	27
Listing 26	Reacting to changes in the radio button group to update the users preference setting.	28
Listing 27	Overwriting the base styling CSS custom properties with the CSS attribute selector on <code>:root</code> for a dark color scheme styling.	29
Listing 28	Considering both the color scheme as well as the high contrast setting leads to another styling.	29
Listing 29	Additional styling based on the custom propperties can be added with the <code>@container</code> rule in combination with the <code>style()</code> query.	30
Listing 30	A custom color contrast function that adjusts the lightness of the resulting black or white and allows to mix back in some parts of the original color.	31
Listing 31	Extracting the <code>RGBA</code> values from any color in any format through the <code><canvas></code> element.	32

List of Aids

Artificial Intelligence and Language Models

Tool: Gemini (Google), Version 3 Flash.

Type of Usage: Used for linguistic refinement of drafts and formating mathematical formulas (Relative Luminance).

Extent: Specific paragraphs regarding WCAG luminance calculations were refined using the tool. All technical facts and code were manually verified for accuracy.